Devoir n°1 : Patrons de construction (INF461)

KOA ESSAMA PAULIN BRICE
Matricule : 25I2255

Introduction

Ce rapport présente les deux formes du patron Factory étudiées dans le support : *Factory Method* et *Factory*. L'exemple du cours gère trois produits (A, B et C). La variante demandée ajoute un quatrième produit D. Pour chaque patron, le rapport contient le diagramme de classes de l'exemple du cours puis celui de la version modifiée.

1 Exemple vu en cours : trois produits

Dans les deux cas, le client ne connaît le type concret de produit qu'à l'exécution.

1.1 Factory Method

Une seule classe `FabriqueProduit` reçoit une catégorie et crée `ProduitA`, `ProduitB` ou `ProduitC`. Le client dépend de l'interface `Produit`.

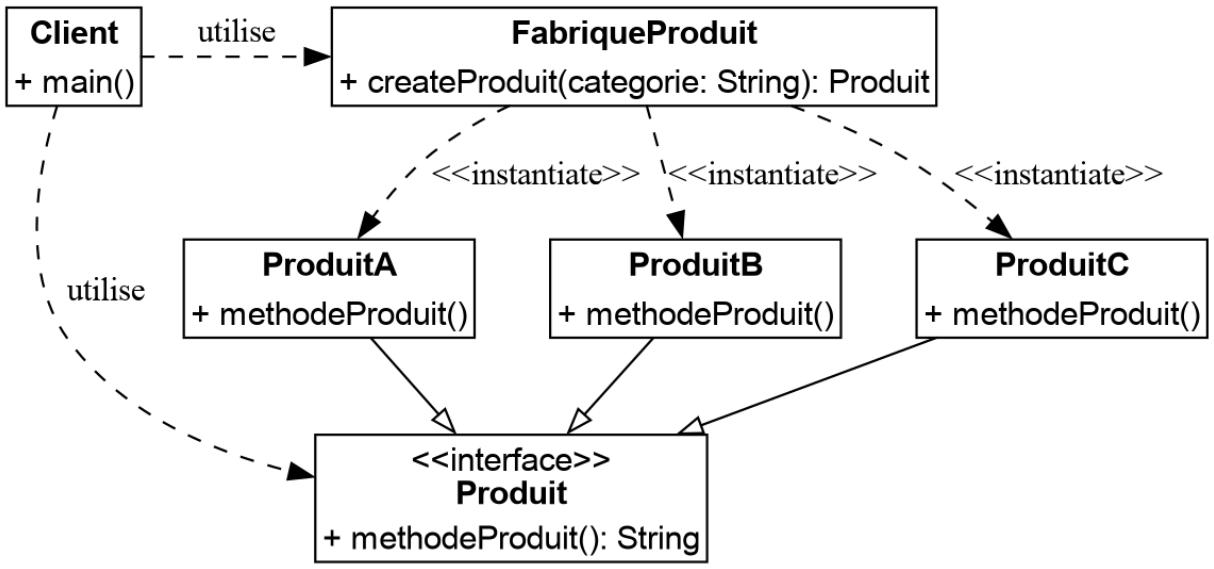


Figure 1: Factory Method – exemple du cours (3 produits)

1.2 Factory

`ProduitFactory` définit la méthode de création. Chaque fabrique concrète crée un seul produit : `ProduitFactoryA` crée `ProduitA`, etc.

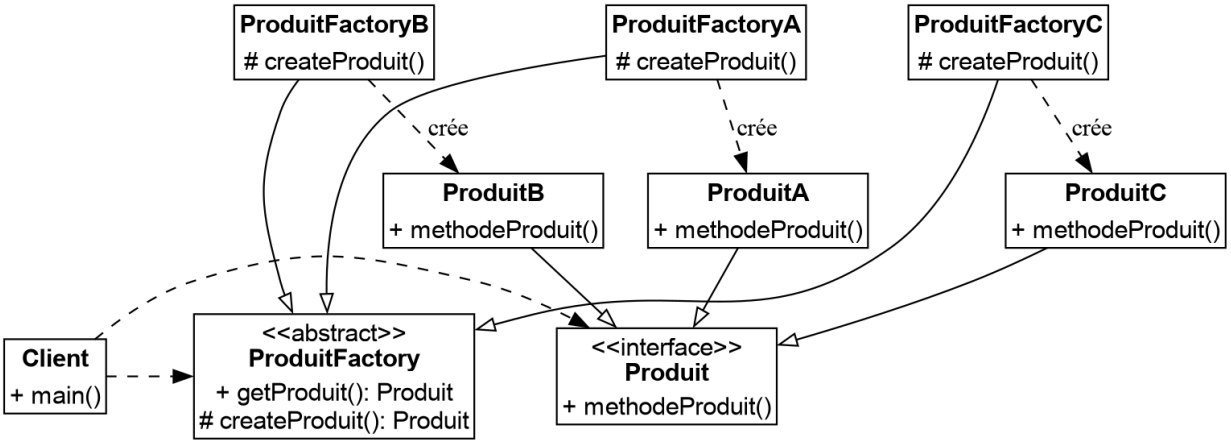


Figure 2: Factory – exemple du cours (3 produits)

2 Exemple modifié : quatre produits

La variante gère maintenant quatre types de produits : A, B, C et D.

2.1 Factory Method modifié

On ajoute `ProduitD` et le cas `D` dans `FabriqueProduit`. La fabrique centrale doit donc être modifiée pour connaître le nouveau produit.

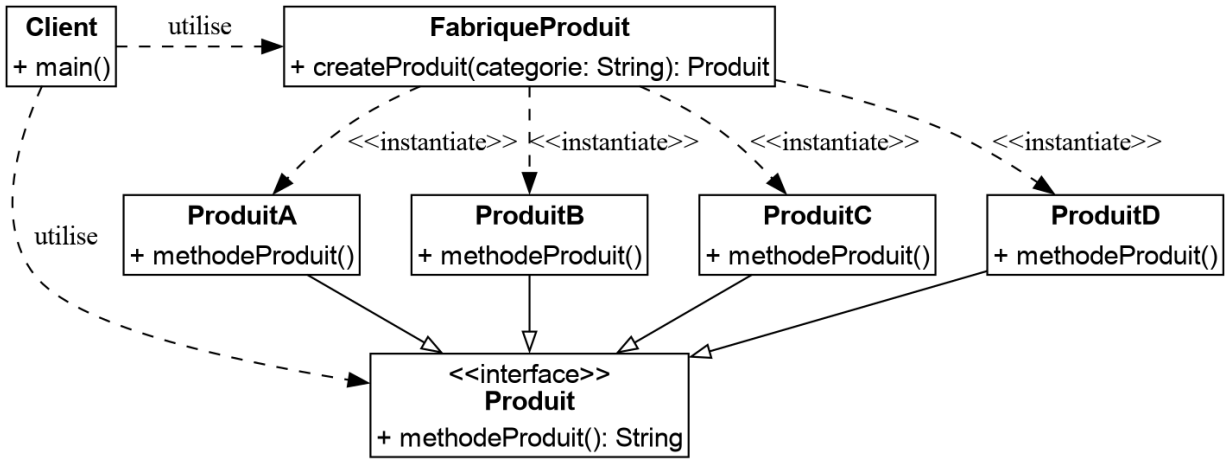


Figure 3: Factory Method – exemple modifié (4 produits)

2.2 Factory modifié

On ajoute `ProduitD` et `ProduitFactoryD`. Les classes déjà présentes ne changent pas ; le client peut utiliser la nouvelle fabrique.

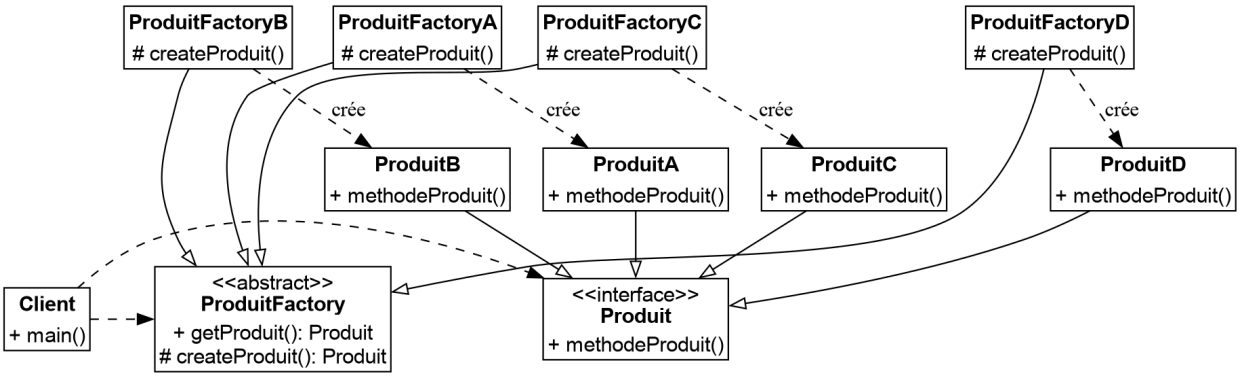


Figure 4: Factory – exemple modifié (4 produits)

3 Implémentation

Le code Java se trouve dans le dossier `src` du projet. Il est organisé en quatre packages :

- `factorymethod.troisproduits` : exemple du cours ;
- `factorymethod.quatreproduits` : variante à quatre produits ;
- `factory.troisproduits` : exemple du cours ;
- `factory.quatreproduits` : variante à quatre produits.

Les diagrammes et leurs fichiers sources sont dans `diagrammes`. Le dépôt Git local est prêt à être publié sur GitHub.

4 Analyse des modèles génériques

Factory Method. Une seule fabrique choisit le produit à l'aide d'un paramètre. Cette solution est simple lorsque le type est connu seulement à l'exécution. Son ajout de `ProduitD` impose toutefois de modifier la classe `FabriqueProduit`.

Factory. Une fabrique abstraite déclare `createProduit()` et chaque sous-classe crée un type concret. L'ajout de D consiste à créer `ProduitD` et `ProduitFactoryD`, sans modifier les fabriques existantes. Cette solution respecte mieux le principe ouvert/fermé, mais nécessite davantage de classes.